Deakin University

Applied Software Engineering

OnTrack Submission

Project Bootstrap: SRS (Group)

Submitted By:
Berlinda Asiedu
s225552263

Tutor:
Hourieh Khalajzadeh

Group Members:

s225793305	Asanga Indunil Sampath	Walakulu Gamage	👨👨👨👨
s223839543	Amoda Rasangi	Senarath	👨👨👨👨
s225552263	Berlinda	Asiedu	👨👨👨👨👨
s225179083	Nishadi Hirunika	Ilandari Devage	👨👨👨👨
s222577043	Sahan Thiranjaya	Devage Don	👨👨👨👨

April 24, 2026

Software Requirements Specification (SRS) for UniConnect

Project Name: **UniConnect Victoria**
Version: **1.0**
Date: **24th April 2026**

1. Introduction

1.1. Document Purpose

This document describes the functional and non-functional requirements for UniConnect Victoria, a web-based platform designed to support university students across Victoria. The document serves as the primary reference for the development team and project stakeholders throughout the design, development, and testing phases of the project. It explains what the system will do, how users will interact with it, and the overall structure of the application.

1.2. Project Scope

UniConnect Victoria is a state-scoped Node.js web application that allows students from different Victorian universities to connect and share academic resources.

The system aims to;

- Improve collaboration between students from different universities
- Provide easy access to shared academic materials
- Increase engagement in student events
- Support networking before entering the workforce

The platform is scoped to Victoria to ensure that the shared academic resources align with local curricula, listed events are geographically reachable by the students, and connections between students are contextually meaningful. The application does not seek to replace official university portals, neither does it provide any direct links to learning management systems (LMS).

1.3. Document Overview

The remainder of this document is organized as follows

- A general description of the system, its context, and its users.
- Key system features and functions
- User roles and system constraints
- Functional and non-functional requirements

1.4. Definitions, Acronyms, and Abbreviations

Term	Meaning

SRS	Software Requirements Specification
F	Functional Requirement
NF	Non-Functional Requirement
UC	Use Case
UI	User Interface
DB	Database
API	Application Programming Interface
Node.js	A cross-platform JavaScript runtime environment
MongoDB	NoSQL database program
RBAC	Role-Based Access Control
EJS	Embedded JavaScript
LMS	Learning Management System

2. Overall Description

2.1. Product Perspective

UniConnect Victoria is a standalone web application. It is developed using Node.js and Express for backend services EJS for frontend rendering and MongoDB for data storage. This system complements existing university portals and LMSs. The system integrates multiple features such as resource sharing, event management, forums, and networking into one platform.

2.2. Product Functions

At a high level, UniConnect provides the following main functions:

- User registration and login using university email and profile management
- Upload and browse academic resources
- Create, discover, view and register for campus events across Victoria
- Post, reply to, and search for topics for discussion in forum
- Search and connect with students from other Victorian universities
- Content and platform management using an administrator panel

2.3. User Classes and Characteristics

The system will have three key users: Students, Event Organizers/ Resource Contributors, and System Administrators.

User Class	**Description**	**Characteristics**
Student	Any student from a Victorian university (undergraduate or postgraduate)	Serves as the primary user. Uploads resources and events, registers for events, sends and replies to posts in the forum, and connects with other students.
Guest (Event Organizer/Resource Contributor)	Verified event organizers and academic professionals	Upload events and resources for student access
System Administrator	A designated technical person trusted with elevated privileges to manage user engagement and platform content	Reviews posts and uploaded resources, approves events before making them public, ensures compliance with set policies and regulations.

2.4. System Constraints

The following constraints are imposed on the system, and the development team must strictly comply with them as they cannot be negotiated as part of the design;

- The system must support major web browsers (Google Chrome, Safari, Firefox, Edge).
- The system must be accessible via both mobile and personal computers.

- The database must be MongoDB.
- The application must be built using Node.js.
- The styling framework must be Materialize CSS. Additional CSS is permitted but must not conflict with Materialize.
- The system must comply with data privacy regulations.
- Students must sign up using their verified university email addresses, ending in .edu or .edu.au.
- The system must integrate with third-party APIs and websites for event booking and registration, such as Eventbrite.

2.5. Assumptions and Dependencies

- Users will have internet access and a web browser
- All students have university email addresses
- Users will be based in Victoria and would be attending universities in Victoria
- Students will responsibly share and upload educational resources to the resource library.

3. Specific Requirements

3.1. External Interfaces

- User Interface:

The application shall provide the following primary pages which will be accessible through a navigation menu.

- Home/Dashboard: Personalized feed showing recently uploaded resources, upcoming events, and recent forum activity.
- Resource Library: A catalog of resources that can be searched and filtered. Contains an upload form for users who have been verified.
- Events Board: A list of upcoming events and a form for submitting a new event.
- Forum: A page for student discussions on topics, listed by unit code or subject area.
- Networking Directory: A searchable directory of registered students, with filters to help users find relevant peers and send networking requests.
- User Profile: Displays a user's university, name, email, connections, uploaded resources, events etc.
- Admin Panel: Dashboard that is exclusive to moderators for handling submitted content, approving events, and managing accounts.

- Hardware Interfaces:
- Compatible with devices supporting browsers (e.g. laptops, mobile phones, tablets, computers)

- Software Interfaces:
- Integration with third-party event booking sites
- MongoDB - All CRUD operations on users, resources, events, forum posts, and connections are performed through Mongoose models and schemas.
- Express will be used to manage server-side interactions
- Email based authentication using university email domain

- Communication Interfaces:
- The application communicates over HTTP for development and HTTPS for production (deployment).
- All form submissions use HTML form POST requests.
- Support RESTful API for third-party integrations, email and push notifications.

3.2. Functional Requirements (FR)

User Registration and Login

- F1 - The system shall allow a new user to register using their university email addresses.
- F2 - The system shall reject registrations with non-university email addresses (gmail, yahoo, etc.).

- F3 - The system shall encrypt all user passwords before storing them into the database.
- F4 - Users shall be able to log into the system securely.
- F5 - The system shall authenticate user credentials before authorizing access to the application.
- F6 - Users shall be able to reset their passwords.
- F7 - The system shall allow users to view and update their user profiles.
- F8 - Users shall be able to log out and terminate their active session.

Events Board

- F9 - The system shall display a list of approved upcoming events.
- F10 - The system shall allow users to filter/search for events using certain criteria including event type, location, date.
- F11 - The system shall allow verified users to register to attend events.
- F12 - The system shall require information such as event name, location, organizers, type, date, and registration details to be uploaded to the events board.
- F13 - The system shall allow users to cancel their booking for an event.

Resource Library

- F14 - The system shall allow verified users to upload a resource, indicating the title, description, unit code, resource type, and a URL or file upload.
- F15 - The system shall display a list of all approved resources in a chronological manner.
- F16 - The system shall allow users to search for resources by their title, unit code, and resource type.
- F17 - The system shall approve uploaded resources before being made public and accessible to other users.
- F18 - The system shall allow users to rate a resource.
- F19 - The system shall allow uploaders to edit or delete their own uploaded resources.

Discussion Forum

- F20 - The system shall display a list of forum topics in the subject area and unit code.
- F21 - Users shall be able to create forum posts and create threads under a single post.
- F22 - Users shall be able to reply to posts.
- F23 - Users shall be able to search forums by topic or subject area.
- F24 - Users shall be able to edit or delete their own posts.
- F25 - Users shall be able to like or dislike (flag) posts.

Networking Directory

- F26 - The system shall display a searchable directory of all verified students.
- F27 -The system shall allow users to search for and filter users by name, university, major, nationality, etc.

- F28 - Users shall be able to send connection requests to other verified students.
- F29 - Users shall be able to send and receive direct messages to their connections.
- F30 - Users shall be able to edit and delete their sent messages after a period of time.
- F31 - Users shall receive notifications when they receive messages and connection requests.

Admin Functions

- F32 - Admin users shall be able to manage users, remove inappropriate resources and posts, and monitor platform activity.
- F33 - Admin panels shall be restricted to authorized users only.
- F34 - The system shall display a list of pending event submissions awaiting review and approval.
- F35 - The system shall display a list of flagged resources for rejection.
- F36 - Admin shall be able to approve or reject event and resource submissions.
- F37 - Admin shall be able to deactivate user accounts upon violation of platform policies.

3.3. Non-Functional Requirements

Performance

- NF1 - The system should load all main pages (dashboard, resources, events, forum) within 2-3 seconds under normal conditions
- NF2 - The backend system shall support at least 500 concurrent active users during peak periods without significant degradation in response time
- NF3 - Database queries like search should return results within 2 seconds
- NF4 - The platform should handle increasing data efficiently

Security

- NF5 - User authentication must be required to access important features
- NF6 - Passwords must be securely stored
- NF7 - Only users with university verified email addresses can register
- NF8 - Admin functionalities must be restricted to authorized users only

Usability

- NF9 – At least 85% of test users shall complete core navigation tasks (login, accessing resources, viewing events) without assistance during usability testing.
- NF10 – Key user actions (login, uploading a resource, registering for an event) shall require no more than 4 steps from the main dashboard.

- NF11 - The system shall render correctly on devices with screen widths of 320px (mobile), 768px (tablet), and 1024px+ (desktop), with no critical layout issues detected in responsive testing tools.
- NF12 - The system should provide clear feedback messages (e.g.: success or error messages) within 1 second of user action, and 100% of interactive components shall include feedback messaging verified through UI testing.

Reliability

- NF13 - The system should be available 24/7 with minimal downtime
- NF14 - Regular backups should be maintained to prevent data loss

Scalability

- NF15 - The system should be able to handle and support increasing numbers of up to 20,000 users without requiring any major redesign.
- NF16 - The system must accommodate a 30% annual growth in user traffic. A minimum of 1,000 more resources and events must be supported by the system database.

Compatibility

- NF17 - The system shall be compatible with the latest versions of major browsers.
- NF18 - The system shall support both light mode and dark mode for accessibility.
- NF19 - The system shall work on devices running iOS, Android, Windows, and macOS.

Testability

- NF20 - At least 75% of the backend logic must be supported by automated unit tests.
- NF21 - Integration testing for all significant workflows (login, forum engagement, and event booking) must be supported by the system.
- NF22 - Prior to deployment, the system will offer a staging environment for testing new features.

4. Supporting Information

4.1. System Architecture Diagram

The UniConnect Victoria backend system follows a simple layered architecture to ensure the platform supports all functional and non-functional requirements effectively, including security, scalability, maintainability, and performance.

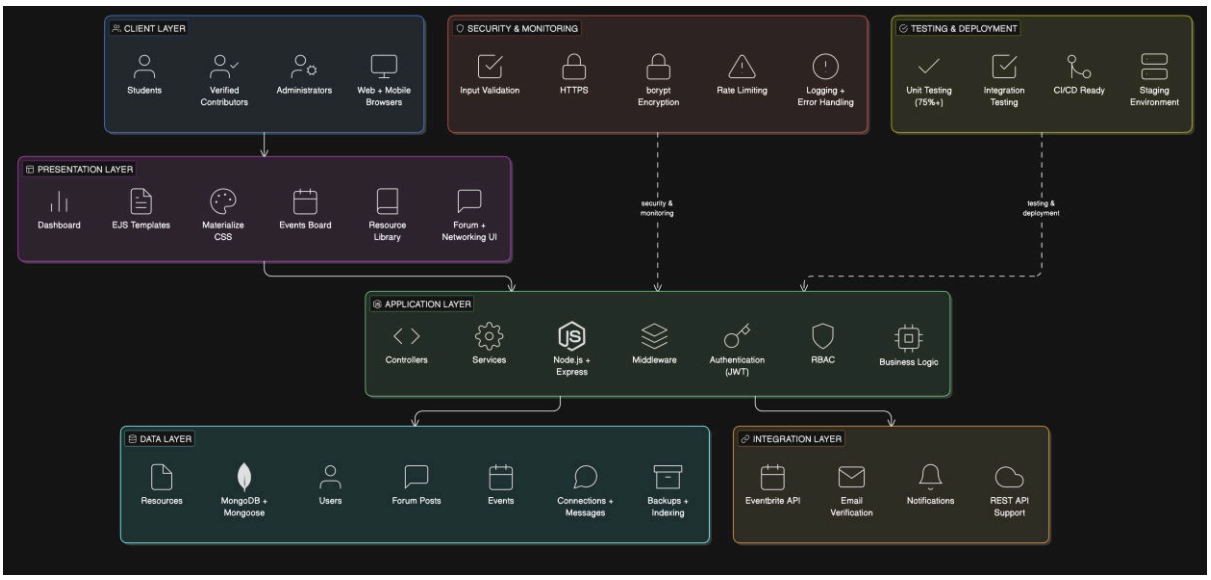

Figure 1: System Architecture Diagram for UniConnect

4.1.1. Client Layer

The Client Layer represents the end users of the system, including Students, Verified Contributors, and System Administrators. Users access the platform through web browsers on desktop and mobile devices using supported browsers such as Chrome, Safari, Firefox, and Edge.

This layer is responsible for sending user requests and receiving system responses.

4.1.2. Presentation Layer

The Presentation Layer handles the user interface of the system using EJS templates and Materialize CSS.

It includes:

- Dashboard
- Resource Library
- Events Board
- Discussion Forum
- Networking Directory
- User Profile
- Admin Panel

This layer ensures users can interact with the platform easily through a responsive and user-friendly interface.

4.1.3. Application Layer

The Application Layer is the core backend of the system built using Node.js and Express.js.
It manages:

- Controllers
- Service Layer
- Middleware
- Authentication using JWT
- Role-Based Access Control (RBAC)
- Business Logic Processing

This layer handles all system operations such as login validation, event approvals, resource uploads, forum interactions, networking requests, and administrator actions.

4.1.4. Security and Monitoring

This supporting layer ensures the system remains secure and reliable.
It includes:

- HTTPS secure communication
- bcrypt password encryption
- Input validation and sanitization
- API rate limiting
- Centralized logging
- Error handling and monitoring

This helps protect the platform against unauthorized access, data breaches, and misuse.

4.1.5. Integration Layer

The Integration Layer manages communication with external systems and third-party services.
It includes:

- Eventbrite API integration
- University email verification services
- Email and notification services
- REST API support for future expansion

This layer allows the platform to interact safely with external services while maintaining system reliability.

4.1.6. Data Layer

The Data Layer uses MongoDB with Mongoose for database management.
It stores and manages:

- User accounts

- Academic resources
- Events
- Forum posts
- Connections and direct messages

It also includes database indexing and backup mechanisms to improve performance and ensure data reliability.

4.1.7. Testing and Deployment

This layer supports system quality and stable deployment.
It includes:

- Automated unit testing (minimum 75%)
- Integration testing
- Staging environment
- CI/CD readiness

This ensures that new features are tested properly before deployment and helps maintain system stability.

4.2. User Stories

- US1 – As a student, I want to sign up using my university email so that I can get access to academic resources.
- US2- As a student, I want to log into the platform with my email address and password so that I can access my dashboard.
- US3 – As a student, I want to be able to update my profile so that it is reflective of my current academic background.
- US4 – As a student, I want to upload articles and research papers with a unit code and subject area so that other students studying the same unit can find them easily.
- US5 – As a student, I want to download or access a resource available in the system so that I can use it for my studies.
- US6 – As a student, I want to filter events by type (academic, career, social) so that I find events relevant to my goals.
- US7 – As a student, I want to RSVP to an event so that the organizer knows that I am attending.
- US8 – As a student, I want to search and filter a directory of registered students so that I can find relevant peers and send networking requests.
- US9 – As a system administrator, I want to review flagged resources from my dashboard so that I can delete them to maintain content quality.
- US10 – As a system administrator, I want to deactivate a user account that has violated platform policies so that the community remains safe.
- US11 – As a system administrator, I want to approve or reject pending upcoming events so that only relevant and appropriate events are published.

- US12 – As a campus event organizer, I want to upload my upcoming events to UniConnect so that more students outside my university can register and attend.

4.3. Use Case Summary

The following table summarizes the primary use cases for the systems and maps them with the corresponding functional requirements.

UC 1 — Register and Log In Using University Email & Manage Profile

Description This use case describes how a user registers and logs in using their university email address and manages their password and profile information.
Primary Actor: User (Student or Staff) **Secondary Actor:** Authentication Service
Basic Flow 1. The user selects the option to register or log in. 2. The system prompts the user to enter their university email address and password. 3. The system sends the credentials to the Authentication Service for verification. 4. The service validates the email domain and credentials. 5. The user successfully logs in and accesses their profile. 6. The user updates their password or profile details. 7. The system saves the updated information and confirms the changes.
Alternate Flow A1 – Invalid Email Domain 1. The user enters a non-university email address. 2. The system displays an error indicating that only university emails are allowed.
Alternate Flow A2 - Incorrect Password 1. The user enters an incorrect password. 2. The system displays an error and offers a password reset option.
Functional Requirements - F1 - F8

UC 2 — Explore Events Library and RSVP to an Event

Description This use case describes how a user browses the events library and RSVPs to an event.
Primary Actor: User (Student or Staff) **Secondary Actor:** Event Management Service
Basic Flow 1. The user navigates to the events library. 2. The system displays a list of available events. 3. The user selects an event to view details. 4. The system retrieves event information from the Event Management Service. 5. The user selects the option to RSVP. 6. The system records the RSVP and confirms the user's attendance.
Alternate Flow A1 – Event at Capacity 1. The user attempts to RSVP to a full event.

2. The system displays a message indicating that no more spots are available.
Functional Requirements - F9 - F13

UC 3 — Explore Resource Library and Access a Document

Description This use case describes how a user browses the resource library and accesses a document.
Primary Actor: User (Student or Staff) **Secondary Actor:** Resource Storage Service
Basic Flow 1. The user opens the resource library. 2. The system displays categories and available documents. 3. The user selects a document to view. 4. The system requests the file from the Resource Storage Service. 5. The service returns the document. 6. The system opens the document for the user.
Alternate Flow A1 – Restricted Document 1. The user selects a document requiring special permissions. 2. The system displays a message indicating restricted access.
Functional Requirements - F14 - F16

UC 4 — Search & Interact with Networking Directory

Description This use case describes how a user searches the networking directory to find students and sends connection requests.
Primary Actor: User (Student) **Secondary Actor:** User Directory Service
Basic Flow 1. The user opens the networking directory. 2. The system displays search and filter options. 3. The user enters search criteria (e.g., course, skills, interests). 4. The system retrieves matching profiles from the User Directory Service. 5. The user selects a profile to view details. 6. The user sends a connection request. 7. The system records the request and notifies the recipient.
Alternate Flow A1 – No Matching Results 1. The system finds no profiles matching the search criteria. 2. The system displays a message indicating no results are found.
Functional Requirements - F26 - F31

UC 5 — Review and Approve Events (Admin)

Description This use case describes how an administrator reviews submitted events and approves them for publishing.
Primary Actor: Administrator **Secondary Actor:** Event Management Service
Basic Flow 1. The administrator opens the event approval dashboard. 2. The system displays a list of pending event submissions. 3. The administrator selects an event to review. 4. The system retrieves event details from the Event Management Service. 5. The administrator approves the event. 6. The system updates the event status and publishes it to the events library.
Alternate Flow A1 – Event Rejected 1. The administrator rejects the event. 2. The system notifies the event creator with feedback.
Functional Requirements - F34, F36

UC 6 - Use Case 6 — Deactivate User Accounts for Policy Violations

Description This use case describes how an administrator deactivates user accounts that violate platform policies.
Primary Actor: Administrator **Secondary Actor:** User Account Service
Basic Flow 1. The administrator accesses the user management panel. 2. The system displays a list of user accounts. 3. The administrator selects a user flagged for policy violations. 4. The system retrieves account details from the User Account Service. 5. The administrator selects the option to deactivate the account. 6. The system updates the account status and confirms deactivation.
Alternate Flow A1 – Insufficient Permissions 1. A non-admin user attempts to access account deactivation features. 2. The system denies access and displays an authorization error.
Functional Requirements - F37

4.4. References

1. IEEE, “IEEE Recommended Practice for Software Requirements Specifications,” IEEE Std 830-1998, pp. 1–40, Oct. 1998, doi: 10.1109/IEEESTD.1998.88286.

2. ISO/IEC/IEEE, Systems and Software Engineering - Life Cycle Processes - Requirements Engineering (ISO/IEC/IEEE 29148:2018), Geneva, Switzerland: ISO, 2018.

3. A. Bach and F. Thiel, “*Collaborative Online Learning in Higher Education—Quality of Digital Interaction and Associations with Individual and Group-Related Factors*,” **Frontiers in Education**, vol. 9, Nov. 2024.

4. N. Su, A. A. Mamun, M. N. H. Reza, Q. Yang, and M. M. Masud, “*Unveiling the Nexus Between Quality and Student Engagement in Web-Based Collaborative Learning Systems*,” **Education and Information Technologies**, vol. 29, pp. 23717–23752, May 2024

5. Member Contributions

Nishadi Ilandari Devage	Functional and Non-Functional Requirements
Berlinda Asiedu	Product Overview and Functional Requirements
Asanga Indunil Walakulu Gamage	System Architecture Diagram (4.1) and Overview / Validate functional non-functional requirements
Amoda Senarath	Use Cases and User Stories
Sahan Thiranjaya Devage Don	User Stories and System Architecture Overview